

PRACTICA: 2

USO DE MATRICES, VECTORES Y ARCHIVOS EN C++



shutterstock.com • 1873265248

1.INTRODUCCION

En esta practica se hace un sistema en C++ capaz de procesar datos obtenidos de sensores de una pinza robotica como la deformacion estructural y las fuerzas en los dedos izquierdo y derecho.

Tambien implementamos un script en bash que automatiza el funcionamiento del programa en segundo plano y controlando su finalizacion.

En esta practica usamos tanto la gestion de procesos como la ejecucion en segundo plano y la automatizacion mediante scripts.

2.OBJETIVOS

- Leer datos dese un archivo de texto
- Almacenar informacion en matrices
- Separa informacion en arrays independientes
- Calcular la media de los sensores
- Analizar la estabilidad del agarre
- Generar un archivo de resultados
- Compilar el programa en C++ y generar un ejecutable
- Ejecucion mediante bash
- Usar Git para el control de versiones

3.MARCO TEORICO

A continuacion una serie de conceptos que hemos tenido en cuenta para poder realizar esta practica:

- Procesos: es un programa en ejecucion con su propio espacio de memoria. En los sistemas operativos, varios procesos pueden ejecutarse de manera simultanea siempre y cuando este gestionado por el planificador del sistema. En esta practica el programa c++ se ejecuta como proceso independiente, lanzado desde el script bash.
- Ejecucion en segundo plano: nos permite que un proceso se ejecute sin bloquear la terminal mediante el operador & ./pinza & por ejemplo, permite que el script continúe su ejecucion sin esperar que el programa termine.
- Identificador de proceso (PID): cada proceso tiene un identificador unico PID que permite gestionar el proceso → consultarlo, pausarlo o finalizarlo. PID=\$! Obtiene el PID del ultimo proceso lanzado en segundo plano.
- Gestion de procesos: usamos varios comandos que permiten controlar el ciclo de vida del programa.
Ps: muestra procesos activos
Wait: espera a que un proceso termine
Kill: finaliza un proceso
- Entrada y salida de archivos en C++: permite almacenar datos para despues procesarlos.
Ifstream: para leer archivos
Ofstream: para escribir archivos
- Arrays y estructura de datos: usamos arrays para almacenar los datos de sensores
IDs
Valores de galga
Fuerza de izquierda
Fuerza de derecha
Tambien se usa una matriz para organizar los datos de forma estructurada.
- Procesamiento de datos: se usan operaciones basicas para analizar el comportamiento del sistema.
Recorrido de arrays
Acumulacion de valores
Calculo de medias
- Analisis de estabilidad: el criterio de estabilidad viene dado en la diferencia de fuerzas.
 $|Fuerza_{izq} - fuerza_{der}| > 0.15 \rightarrow$ permite detectar desequilibrios en la pinza robotica

- Automatizacion con scripts: usar scripts bash nos permite automatizar tareas repetitivas como:
Ejecutar programas
Compilar código
Controlar procesos
- Control de versiones con Git: git nos deja manejar cambios en el código facilitándonos trabajo en equipo, historial de modificaciones y recuperar versiones anteriores.

4.DISEÑO Y METODOLOGIA

Hemos dividido la practica en dos partes:

Parte 1: Programa en C++

Encargado de:

- Leer los datos del archivo
- Procesarlos
- Generar resultados

Parte 2: script en bash

Encargado de:

- Compilar el programa
- Ejecutarlo en segundo plano
- Controlar su ejecucion

Decisiones de diseño:

- Usar arrays
- Separacion del trabajo (uno se encargo del C++ y otro del bash)
- Automatizacion con el script
- Uso de git para el control de versiones.

Ademas de todo esto ibamos haciendo pruebas continuas tras cada modificacion, lo que permitio detectar errores como:

- Problemas ejecucion del script
- Errores de compilacion
- Fallos en el formato de salida

5.IMPLEMENTACION

Lectura de datos

Como hemos mencionado antes, usamos ifstream para leer archivos:

```
/ int main() {  
    //abrir un archivo  
    ifstream archivo("datos_pinza.txt");
```

Los datos se almacenan en arrays:

```
float datos[100][3];  
int ids[100];  
float galga[100];  
float fuerza_izq[100];  
float fuerza_der[100];  
int n=0; //contador de datos
```

Calculo de medias

Se recorren los arrays acumulando los valores y dividiendo por el numero de muestras:

```
float media_galga=suma_galga/n;
```

Analisis de estabilidad

Se calcula la diferencia:

```
float diferencia=abs(fuerza_izq[i]-fuerza_der[i]);
```

Si es mayor de 0,15 es INESTABLE

Script bash

Compilacion:

```
~/practica2-so/practica2 $ g++ main.cpp -o pinza
```

Ejecucion en segundo plano:

```
./pinza & #ejecuta el programa en segundo plano(el & permite seguir ejecutando el script)
```

Obtencion del PID:

```
PID=$! #guarda el ID del proceso que se ha lanzado en segundo plano
```

Espera del proceso:

```
wait $PID #espera que el proceso termine antes de continuar con el script
```

Comprobacion:

```
if ps -p $PID > /dev/null #comprueba si el proceso sigue activo (ps lista procesos y -p busca por PID)
```

Finalizacion:

```
kill $PID #finaliza el proceso manualmente con PID
```

6.RESULTADOS Y PRUEBAS

El programa se ejecuta correctamente mostrando:

- Datos leídos
- Medias calculadas
- Clasificación de estabilidad

```
Compilando...
Ejecutando...
PID: 8097
Datos leídos:
ID: 5118 Galga: 0.24 Fuerza izquierda: 1.796 Fuerza derecha: 1.752
ID: 6574 Galga: 0.22 Fuerza izquierda: 1.595 Fuerza derecha: 1.676
ID: 18852 Galga: 0.229 Fuerza izquierda: 1.574 Fuerza derecha: 1.617
ID: 1202 Galga: 0.251 Fuerza izquierda: 1.841 Fuerza derecha: 1.835
ID: 13342 Galga: 0.247 Fuerza izquierda: 1.806 Fuerza derecha: 1.79
ID: 14167 Galga: 0.262 Fuerza izquierda: 1.921 Fuerza derecha: 1.973
ID: 6178 Galga: 0.234 Fuerza izquierda: 1.68 Fuerza derecha: 1.72
ID: 10733 Galga: 0.263 Fuerza izquierda: 1.968 Fuerza derecha: 1.883
ID: 8593 Galga: 0.224 Fuerza izquierda: 1.629 Fuerza derecha: 1.719
ID: 19963 Galga: 0.232 Fuerza izquierda: 1.722 Fuerza derecha: 1.673
ID: 10490 Galga: 0.239 Fuerza izquierda: 1.868 Fuerza derecha: 1.833
ID: 10041 Galga: 0.241 Fuerza izquierda: 1.683 Fuerza derecha: 1.749
```

Archivo generado:

```
Medias:
Galga = 0.236
Fuerza izquierda = 1.71876
Fuerza derecha = 1.72284
```

En la siguiente imagen se puede ver la ejecución del programa desde el script bash, donde se visualizan los datos procesados y la clasificación de estabilidad de cada muestra:

```
Evaluar la estabilidad:
5118 estable
6574 estable
18852 estable
1202 estable
13342 estable
14167 estable
6178 estable
10733 estable
8593 estable
19963 estable
10490 estable
```

Ademas en la siguiente figura se representa el contenido del archivo de salida generado por el programa, donde estan los datos originales, las medias calculadas y la clasificaion de estabilidad usando cat resultado_pinza.txt

```
anderzp in @anderHP in :~/practica2-so/practica2 $ git pull
Ya está actualizado.
anderzp in @anderHP in :~/practica2-so/practica2 $ cat resultado_pinza.txt
Resultados de pinza:
datos:
ID:5118 Galga:0.24 Fuerza izquierda:1.796 Fuerza derecha:1.752
ID:6574 Galga:0.22 Fuerza izquierda:1.595 Fuerza derecha:1.676
ID:18852 Galga:0.229 Fuerza izquierda:1.574 Fuerza derecha:1.617
ID:1202 Galga:0.251 Fuerza izquierda:1.841 Fuerza derecha:1.835
ID:13342 Galga:0.247 Fuerza izquierda:1.806 Fuerza derecha:1.79
ID:14167 Galga:0.262 Fuerza izquierda:1.921 Fuerza derecha:1.973
ID:6178 Galga:0.234 Fuerza izquierda:1.68 Fuerza derecha:1.72
ID:10733 Galga:0.263 Fuerza izquierda:1.968 Fuerza derecha:1.883
ID:8593 Galga:0.224 Fuerza izquierda:1.629 Fuerza derecha:1.719
ID:19963 Galga:0.232 Fuerza izquierda:1.722 Fuerza derecha:1.673
ID:10490 Galga:0.239 Fuerza izquierda:1.868 Fuerza derecha:1.833
ID:10041 Galga:0.241 Fuerza izquierda:1.683 Fuerza derecha:1.749
ID:8863 Galga:0.216 Fuerza izquierda:1.509 Fuerza derecha:1.517
ID:1099 Galga:0.238 Fuerza izquierda:1.792 Fuerza derecha:1.905
ID:15736 Galga:0.227 Fuerza izquierda:1.558 Fuerza derecha:1.569
ID:19230 Galga:0.23 Fuerza izquierda:1.607 Fuerza derecha:1.667
ID:19696 Galga:0.206 Fuerza izquierda:1.528 Fuerza derecha:1.531
ID:6658 Galga:0.23 Fuerza izquierda:1.576 Fuerza derecha:1.788
ID:12773 Galga:0.21 Fuerza izquierda:1.522 Fuerza derecha:1.465
ID:3575 Galga:0.244 Fuerza izquierda:1.817 Fuerza derecha:1.86
ID:4932 Galga:0.239 Fuerza izquierda:1.779 Fuerza derecha:1.838
ID:12213 Galga:0.234 Fuerza izquierda:1.60 lerecha:1.651
ID:15955 Galga:0.197 Fuerza izquierda:1.44 lerecha:1.523
ID:12533 Galga:0.21 Fuerza izquierda:1.583 Fuerza derecha:1.569
```

```
Medias:
Galga = 0.236
Fuerza izquierda = 1.71876
Fuerza derecha = 1.72284
Evaluar la estabilidad:
5118 estable
6574 estable
18852 estable
1202 estable
13342 estable
14167 estable
6178 estable
```

Tambien mencionar que hemos aprendido a ocultar archivos no deseados en nuestro git para una mejor presentacion mediante gitignore:

```
nano .gitignore
git rm --cached pinza
```


7.DISCUSION

Durante la elaboracion de nuestra practica nos encontrado con varios problemas:

- Errores de ejecucion del script bash, debido al formato del archivo CRLF y LF

```
lukks@lukksHP:~/practica2-S0/practica2$ nano script.sh
lukks@lukksHP:~/practica2-S0/practica2$ chmod +x script.sh
lukks@lukksHP:~/practica2-S0/practica2$ ./script.sh
bash: ./script.sh: cannot execute: required file not found
```

- Falta del ejecutable pinza, debido a que no se compilo el programa previamente:

```
bash: ./script.sh: cannot execute: required file not found
lukks@lukksHP:~/practica2-S0/practica2$ ls
build  datos_pinza.txt  main.cpp  resultado_pinza.txt  script.sh

build  datos_pinza.txt  main.cpp  pinza  resultado_pinza.txt  script.sh
```

```
lukks@lukksHP:~/practica2-S0/practica2$ ./script.sh
Compilando...
Ejecutando...
PID: 7171
Datos leidos:
ID:5118Galga:0.24Fuerza izquierda:1.796Fuerza derecha:1.752
ID:6574Galga:0.22Fuerza izquierda:1.595Fuerza derecha:1.676
ID:18852Galga:0.229Fuerza izquierda:1.574Fuerza derecha:1.617
```

- Errores basicos en la configuracion del usuario y autentificaiion del token.
- Problemas de formato en la salida que hacian dificil la lectura de resultados:

```
ID:5118Galga:0.24Fuerza izquierda:1.796Fuerza derecha:1.752
ID:6574Galga:0.22Fuerza izquierda:1.595Fuerza derecha:1.676
ID:18852Galga:0.229Fuerza izquierda:1.574Fuerza derecha:1.617
ID:1202Galga:0.251Fuerza izquierda:1.841Fuerza derecha:1.835
ID:13342Galga:0.247Fuerza izquierda:1.806Fuerza derecha:1.79
ID:14167Galga:0.262Fuerza izquierda:1.921Fuerza derecha:1.973
ID:6178Galga:0.234Fuerza izquierda:1.68Fuerza derecha:1.72
ID:10733Galga:0.263Fuerza izquierda:1.968Fuerza derecha:1.883
ID:8593Galga:0.224Fuerza izquierda:1.629Fuerza derecha:1.719
ID:19963Galga:0.232Fuerza izquierda:1.722Fuerza derecha:1.673
ID:10490Galga:0.239Fuerza izquierda:1.868Fuerza derecha:1.833
ID:10041Galga:0.241Fuerza izquierda:1.683Fuerza derecha:1.749
```

Estos problemas los fuimos solucionando de las siguientes maneras:

- Conversion de formato de archivos:

```
lukks@lukksHP:~/practica2-S0/practica2$ dos2unix script.sh
dos2unix: converting file script.sh to Unix format...
lukks@lukksHP:~/practica2-S0/practica2$ chmod +x script.sh
lukks@lukksHP:~/practica2-S0/practica2$ ./script.sh
```

- Editando el main.cpp para que la impresion se vea mas limpia:

```
ID: 5118 Galga: 0.24 Fuerza izquierda: 1.796 Fuerza derecha: 1.752
ID: 6574 Galga: 0.22 Fuerza izquierda: 1.595 Fuerza derecha: 1.676
ID: 18852 Galga: 0.229 Fuerza izquierda: 1.574 Fuerza derecha: 1.617
ID: 1202 Galga: 0.251 Fuerza izquierda: 1.841 Fuerza derecha: 1.835
ID: 13342 Galga: 0.247 Fuerza izquierda: 1.806 Fuerza derecha: 1.79
ID: 14167 Galga: 0.262 Fuerza izquierda: 1.921 Fuerza derecha: 1.973
ID: 6178 Galga: 0.234 Fuerza izquierda: 1.68 Fuerza derecha: 1.72
```

8.CONCLUSIONES

La practica nos ha enseñado a usar tanto conocimientos de C++ como conceptos que sabiamos de sistemas operativos como la gestion de procesos y automatizacion.

Hemos conseguido:

- Procesar datos reales
- Analizar su comportamiento
- Automatizar la ejecucion
- Gestionar versiones mediante Git

Ademas se podria decir que hemos aprendido a resolver ciertos errores ya presentados en el punto anterior que no creiamos necesitar al empezar la practica, ademas de conseguir todos los objetivos.

9.MANUAL DE EJECUCION

A continuacion un pequeño manual para poder ejecutar el trabajo de manera adecuada.

Compilar manualmente:

G++main.cpp -o pinza

Ejecutar el script:

Chmod +x script.sh → dar los permisos

./script.sh

Y con esto ya podras ver como van apareciendo los datos, que ademas se van guardando el txt que se encuentra en nuestro repositorio de Github.

Para comprobar que funciona correctamente de forma mas rapida tambien puedes usar:

```
cat resultado_pinza.txt
```

10.REPOSITORIO GIT

Link al repositorio:

Contenido:

Main.cpp → programa principal

[script.sh](#) → automatizacion

Datos_pinza.txt → entrada

Resultado_pinza.txt → salida

Hemos realizado commits progresivos y limpieza del repositorio mediante .gitignore

11.BIBLIOGRAFIA

Conversion:

<https://manpages.ubuntu.com/manpages/trusty/es/man1/dos2unix.1.html>

Como funciona .gitignore:

<https://www.datacamp.com/es/tutorial/gitignore>